Measuring Schedule Loss in Self-Scheduled Frequency-Hopping Links

Rishit Sinha 24BEC0529
Electronics and Communications
Vellore Institute of Technology
Vellore, India
rishit.sinha2024@vitstudent.ac.in

Rupsa Mittra 24BEC0176
Electronics and Communications
Vellore Institute of Technology
Vellore, India
rupsa.mittra2024@vitstudent.ac.in

Velmurugan T.
Electronics and Communications
Vellore Institute of Technology
Vellore, India
tvelmurugan@vit.ac.in

***Abstract*—A frequency-hopping link that carries its own hop schedule inside its data packets needs neither a synchronised clock nor a control channel, which makes it attractive for low-cost unmanned aerial vehicle (UAV) telemetry. This paper shows that the arrangement couples two failures that are usually independent: losing a packet also loses the schedule. The consequence is measured on a two-node 433 MHz link built from ESP32 microcontrollers and CC1101 transceivers, under channel denial emulated in firmware so that no interference is ever transmitted. Denying one of five channels removed 98 % of traffic rather than the 20 % a per-channel argument predicts, an amplification of 4.9. Adding cyclic rendezvous recovery bounds the damage but makes it *saturate*: delivery becomes independent of how many channels are denied and depends only on the hop distance from the rendezvous channel to the first denied one. A closed-form expression in that single parameter reproduces three measured delivery counts exactly, with no fitted quantity. Transmitter-side clear-channel assessment avoids the channels it can detect as occupied and reduces loss to 2 %, with no false alarms in 101 measurements against a noise floor of −105.3 ± 1.6 dBm. The contribution is a characterisation and a model of a failure mode rather than a new hopping scheme. Reliability improves when interference affects part of the hop set; no claim is made against wideband denial.**

***Index Terms*—Channel blacklisting, clear channel assessment, frequency-hopping spread spectrum, hop schedule, link reliability, rendezvous recovery, unmanned aerial vehicle telemetry.**

I. Introduction

A small unmanned aerial vehicle (UAV) depends on its telemetry link for state reporting, command upload and, critically, for the failsafe and return-to-home behaviour invoked when something goes wrong. In the low-cost segment that link is almost always a single narrowband channel: a fixed-frequency serial radio bridging the flight controller to a ground station. Any device placing energy on that one channel removes the entire link, and removes it precisely when the aircraft most needs to be told what to do.

Frequency-hopping spread spectrum (FHSS) is the classical answer. The two ends agree on a set of channels and move through them together, so energy confined to one channel can only affect the fraction of traffic sent while the link happens to be there. Adding transmitter-side clear-channel assessment (CCA) sharpens this further: a channel that looks occupied is skipped before it is used, rather than discovered to be unusable after a packet has already been lost on it.

Two design choices follow from the constraint that the radio is a microcontroller-class device with no accurate time reference. First, the hop schedule is *self-scheduled*: each packet names the channel the transmitter will move to next, so neither a shared clock nor a control channel is required. Second, the receiver returns to an agreed rendezvous channel when it has heard nothing for a fixed interval. Both choices are standard, and both are cheap.

Much of the published work on UAV anti-jamming is evaluated in simulation [1], [12], [14], where the schedule is modelled as being known to both ends. On hardware the distribution of that schedule is itself a failure surface, and this paper shows that it is the dominant one. Self-scheduling couples two failures that are normally independent: a packet lost on an unusable channel also carries away the instruction that would have told the receiver where to go next. The link does not degrade in proportion to the fraction of the band removed — it stops.

This paper is therefore a characterisation and a model rather than a new hopping scheme. The contributions are as follows.

1) *Schedule-coupled loss* is identified and measured. Denying one channel of five removed 98 % of traffic rather than 20 %, an amplification of 4.9, and the mechanism is traced to loss of the next-channel instruction rather than to loss of the payload.
2) *Denial saturation* is reported: once rendezvous recovery is present, delivery no longer depends on how many channels are denied, only on the hop distance d from the rendezvous channel to the first denied one. Denying a second channel changed the measured delivery count by zero packets.
3) A closed-form delivery model in the single parameter d is derived from the mechanism and reproduces three measured delivery counts exactly, with no fitted quantity.
4) The resulting design rule is quantified: recovery is necessary but not sufficient, and transmitter-side clear channel assessment, which avoids the affected channel before the schedule is lost on it, takes the same link from 98 % loss to 2 %. Supporting measurements include a noise floor of −105.3 ± 1.6 dBm, a false-alarm rate of 0 in 101, and a channel-plan error that only measurement exposed.

II. Related Work

A. Anti-Jamming for UAV Links

The telemetry protocols in common use offer no protection of their own. MAVLink, surveyed by Koubaa *et al.* [19], transmits unauthenticated and unencrypted by default; MAVSec [17] adds encryption at the cost of latency, and neither addresses availability. Petnga and Xu [25] place jamming within a broader cyber-physical threat model for

UAVs. Surveys of jamming and its countermeasures [16], [23], together with the measurement study of Xu *et al.* [32], establish both how cheaply a narrowband link is removed and how awkward it is to distinguish jamming from ordinary poor reception.

Recent UAV-specific countermeasures are largely learning-based or game-theoretic. Lv *et al.* [24] formulate transmit power as a game against a jammer; Jiao *et al.* [14] shape the trajectory of the aircraft itself; Liu *et al.* [21] select channels from a spectrum waterfall using deep reinforcement learning; and Wang *et al.* [12] switch between intelligent hopping and backscatter. All four are evaluated in simulation and assume computational and sensing resources well beyond a microcontroller-class telemetry radio, which is the design point addressed here.

The most recent work continues in the same direction. Cheng *et al.* [7] apply deep reinforcement learning to fast frequency hopping, and Song *et al.* [10] jointly adapt rate and frequency against a jammer; Xie *et al.* [13] extend the approach to UAV-assisted device-to-device networks. A parallel line of work detects rather than evades: Cibecchini *et al.* [8] classify drone jamming with a lightweight model, Ayegun *et al.* [6] target universal detection across attack types, and Oh and Kang [5] give early warning from link statistics in drone ad-hoc networks. Detection is complementary to the present work: the failure reported here is visible precisely because the link is instrumented packet by packet, but detecting a bad channel is of no use if the act of discovering it destroys the schedule.

B. Hop-Sequence Secrecy Versus Hop-Schedule Distribution

A distinction the literature does not usually draw is central to this paper. Most anti-jamming hopping work improves the *sequence*: how the order of channels is generated, and how hard it is for an adversary to predict. Atheeq *et al.* [1] are representative, deriving the hop order from a chaotic map so that observing past hops does not reveal future ones. Classical treatments of processing gain under partial-band jamming [35], [34] likewise concern the sequence and the spreading factor.

The present work concerns the *distribution* of the schedule instead: given that both ends must agree on where to go next, how is that agreement conveyed, and what happens when the conveying mechanism fails? Sequence secrecy and schedule distribution are orthogonal. A perfectly unpredictable sequence must still be communicated, and if it is communicated in band it inherits the coupling reported in Section V. To the authors' knowledge the cost of that coupling has not previously been measured or modelled.

C. Sensing Before Transmitting

Assessing a channel before using it is long established. Energy detection and its alternatives are surveyed by Yucek and Arslan [29]. Within hopping systems specifically, Bluetooth adaptive frequency hopping classifies channels and removes bad ones from the hop set [30], and Kim *et al.* [33] combine listen-before-talk with adaptive hopping for Bluetooth/WLAN coexistence, which is the closest prior art to the sensing layer used here. Listen-before-talk underpins unlicensed-band coexistence more generally [26], and measured coexistence in shared ISM spectrum is reported in [31] and, more recently and at length, in [9]. Threshold choice remains an active question: He *et al.* [3] set a sliding-window detection threshold adaptively for high-speed hopping signals, where a fixed threshold either misses occupancy or raises false alarms. The present work uses a fixed threshold and reports its measured false-alarm rate rather than tuning it. The sensing mechanism in this paper is therefore not claimed as novel; it is used as the control that establishes the cost of the coupling.

D. Blacklisting in Scheduled Hopping Networks

Blacklisting is well studied in IEEE 802.15.4 time-slotted channel hopping. Kotsiou *et al.* [18] compare distributed and global blacklisting against an experimental dataset, Elsts *et al.* [22] adapt channel selection online, and Cena *et al.* [15] apply probabilistic blacklisting for low-power operation. The setting in all three is a scheduled multi-hop mesh with a network-wide slot number [27], so blacklist state can be disseminated and agreed between nodes. A two-node telemetry link has neither a global clock nor a dissemination path, which is precisely why the in-band scheduling of Section III-C is used, and why its failure mode matters.

How a blacklisted channel is brought back also matters. Jo and Baek [4] show that the remapping used by adaptive fast frequency hopping leaves recovered channels under-used, and propose an explicit restoration step. The cooldown of Section III-E serves the same purpose by the simplest available means: a blacklisted channel is retried after a fixed number of cycles, so a channel that was briefly busy returns to the hop set without any signalling between the two ends.

E. Rendezvous and Schedule Re-Acquisition

Returning to an agreed channel after losing contact is the rendezvous problem of cognitive radio. Lin *et al.* [28] give a jump-stay construction with a guaranteed bound on time to rendezvous, and Tan and Yuan [20] extend blind rendezvous to multi-radio nodes; Chang *et al.* [2] and Asadi Vasfi and Shahgholi Ghahfarokhi [11] give more recent sequences with tighter diversity and time-to-rendezvous guarantees. That literature asks how quickly two nodes *can* meet. The question here is different and, for this failure mode, more consequential: given that they do meet, how much traffic is delivered before the schedule is lost again? The saturation result of Section V follows from that second question.

F. Patent Precedent

The constituent mechanisms have patent precedent and are not claimed here. US 6,169,761 [40] claims, for a frequency-hop system, comparing a received signal strength measurement against a threshold to classify a channel as busy, omitting busy channels from use, and recovering synchronisation through an agreed channel. US 8,165,242 [39] addresses jamming avoidance at the *receiver*, selecting among dwells and

resynchronising, without involving the transmitter in channel selection and without maintaining a blacklist.

G. Position of This Work

Table I sets this paper against its closest neighbour in the literature. The difference is one of contribution type as much as of mechanism: [1] proposes a scheme and shows it works in simulation, whereas this paper measures and models why a widely used arrangement fails on hardware. The two are complementary, in that a chaotic sequence distributed in band would exhibit exactly the coupling reported here.

TABLE I
POSITION RELATIVE TO THE CLOSEST PRIOR WORK

Aspect	**Atheeq *et al.* [1]**	**This work**
Object of study	Hop sequence generation	Hop schedule distribution
Mechanism	Chaotic map	In-band next-channel announcement
Adversary assumed	Predicts the sequence	Renders part of the hop set unusable
Evaluation	Network simulation	Two-node COTS hardware, 433 MHz
Headline output	A scheme that mitigates jamming	A closed-form delivery model, validated
Failure analysis	Not reported	The primary contribution
Denial method	Modelled jammer	Firmware-emulated; nothing transmitted

III. SYSTEM DESIGN

A. Hardware

The system comprises two identical nodes, shown in Fig. 1. Each pairs an ESP32 DevKit v1 [36] with a CC1101 sub-1 GHz transceiver [38] over a four-wire SPI bus at 3.3 V. Table II gives the inventory and the interconnection.

One node runs transmitter firmware and one runs receiver firmware, but the hardware is interchangeable, and both share a single configuration header copied into each sketch at build time. That makes it structurally impossible for the two ends to disagree about the channel plan or the radio parameters.

TABLE II
HARDWARE AND INTERCONNECTION

Item	**Qty**	**Role**
ESP32 DevKit v1	2	Protocol logic, SPI master, logging
CC1101 433 MHz module	2	2-FSK transceiver, RSSI measurement
Quarter-wave antenna	2	One per transceiver
CC1101 pin		*ESP32 pin*
VCC		3.3 V
GND		GND
SCK		GPIO 18
MOSI / SI		GPIO 23
MISO / SO		GPIO 19
CSN / CS		GPIO 5

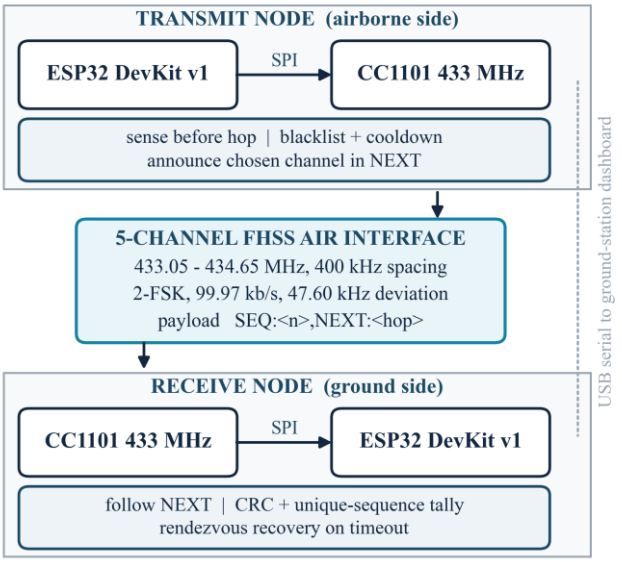


Fig. 1. System architecture. All adaptive behaviour sits at the transmitter, so the receiver firmware is unchanged between the basic and adaptive designs.

B. Physical Layer and Channel Plan

Table III lists the physical-layer configuration, applied identically at both ends. Binary frequency-shift keying is used at 99.97 kb/s with 47.60 kHz deviation, a 16-bit sync word and hardware CRC.

TABLE III
PHYSICAL-LAYER CONFIGURATION

Parameter	Value	Purpose
Modulation	2-FSK	Simple and robust
Deviation	47.60 kHz	Sets occupied bandwidth, (1)
Data rate	99.97 kb/s	Short dwell per packet
Receive bandwidth	270.83 kHz	Passes (1), rejects neighbours
Sync word	0xD391 (16 bit)	16/16 bits must match; MDMCFG2.SYNC_MODE = 2
CRC	Enabled	Corrupt frames dropped in hardware
Transmit power	+10 dBm	Within the 10 mW e.r.p. limit
Hop set size N	5	Five channels, 400 kHz apart
Packet interval	1.2 s	Measured mean

The occupied bandwidth of one transmission follows Carson's rule,

$$B \approx 2(\Delta f + R_b/2) = 2(47.60 + 50.0) \approx 195 \text{ kHz}, \quad (1)$$

where Δf is the peak deviation and R_b the bit rate. The five channel centres are

$$f_i = 433.05 + 0.40\, i \text{ MHz}, \quad i = 0, \ldots, 4, \quad (2)$$

placing all five inside the 433.05–434.79 MHz licence-exempt band [37]. Fig. 2 shows the plan.

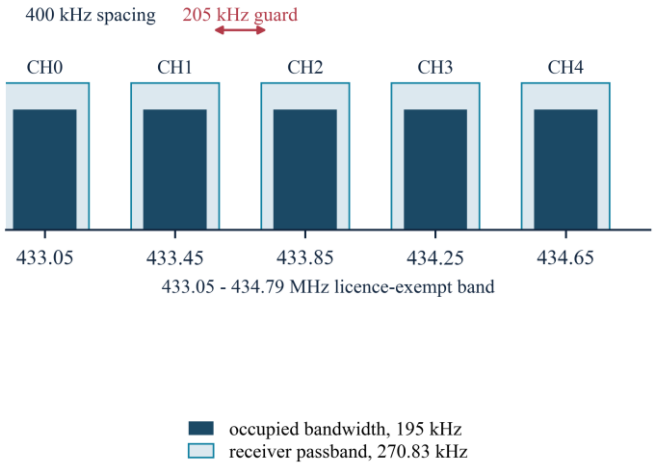


Fig. 2. Channel plan. Each transmission occupies about 195 kHz; the receiver passband is 270.83 kHz, so energy from an adjacent channel falls outside it.

The spacing matters more than it first appears. With 400 kHz spacing the nearest edge of an adjacent transmission sits at

$$\Delta = 400 - B/2 = 400 - 97.5 = 302.5 \text{ kHz} \quad (3)$$

from the wanted centre, against a receiver half-bandwidth of 135.4 kHz. The channels are therefore genuinely independent. Section V-E describes the earlier configuration in which they were not.

Two further quantities follow directly from the modulation. The modulation index, the ratio of peak-to-peak deviation to bit rate, is

$$h = 2\Delta f / R_b = 2(47.60) / 99.97 \approx 0.952, \quad (4)$$

which places the link just below the $h = 1$ point and keeps the two tones separable in a 270.83 kHz passband. The bit duration is

$$T_b = 1 / R_b = 1 / 99.97 \text{ kb/s} \approx 10.0 \text{ µs}. \quad (5)$$

The spread-spectrum processing gain is

$$G_p = W_{ss} / B \approx 1.80 \text{ MHz} / 195 \text{ kHz} \approx 9.2 \text{ (9.6 dB)}. \quad (6)$$

Against a single-channel interferer the more conservative figure is the hop-set size itself,

$$G_{p,\min} = 10 \log_{10} N = 10 \log_{10} 5 = 7.0 \text{ dB}. \quad (7)$$

Neither bounds performance against an interferer covering the whole band. For reference, the thermal noise power in the receiver passband is

$$P_n = -174 + 10 \log_{10} B_{rx} = -174 + 54.3 = -119.7 \text{ dBm}, \quad (8)$$

with B_{rx} = 270.83 kHz; Section V-F compares this with the floor actually measured.

C. In-Band Hop Scheduling

Hopping requires both ends to be on the same channel at the same instant. The usual means are a shared clock or a separate control channel, and a two-node link built from two microcontrollers has neither. The scheme used here carries the schedule inside the data. Every packet has the form

```
SEQ:<sequence number>,NEXT:<hop index>
```

so on receiving a packet the receiver learns both its position in the stream and the channel the transmitter is about to move to. Writing $c_{rx}(k)$ for the channel the receiver is tuned to after packet k, the schedule it follows is the recurrence

$$c_{rx}(k+1) = \text{NEXT}(p_k) \text{ on success}, \; c_{rx}(k) \text{ on loss}. \quad (9)$$

The second branch is the whole problem: a loss does not advance the receiver, so it is left on a channel the transmitter has already left. The transmitter visits every channel once per cycle, so the period at which it revisits any given channel is set by the end-to-end packet cadence T_i rather than by the programmed inter-packet delay T_p = 700 ms. Measured over runs 3, 7 and 8, T_i has a median of 1.200 s and a mean of 1.23 s; the excess over T_p is spent in the blocking driver transmit call and the serial logging that follows it. The revisit period is therefore

$$T_c = N \cdot T_i = 5 \times 1.200 \text{ s} = 6.0 \text{ s}, \quad (10)$$

so a stranded receiver hears the transmitter exactly once every T_c and is deaf for the remaining $N - 1$ transmissions. All timing figures quoted in this paper use the measured T_i, not the programmed T_p. Fig. 3(a) shows the nominal case.

This needs no clock discipline, but it couples data delivery to schedule delivery, and that coupling is the failure surface. If a packet is lost, the receiver loses not only its payload but its instruction for where to go next. The transmitter hops; the receiver does not; the link is gone. Fig. 3(b) shows this, and Section V-B measures it.

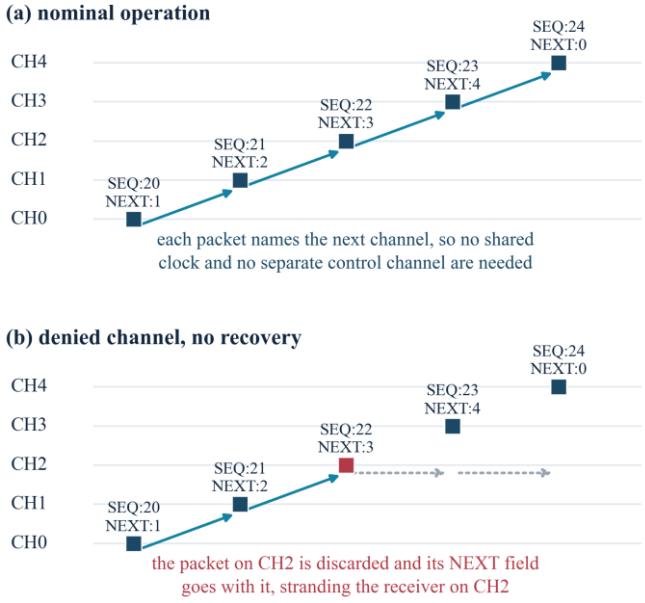


Fig. 3. In-band hop scheduling. (a) Each packet names the channel the transmitter will use next. (b) A packet discarded on a denied channel takes its NEXT field with it, stranding the receiver.

D. Rendezvous Recovery

The mitigation is a cyclic fallback. If the receiver has accepted nothing valid for 2000 ms it retunes to a pre-agreed rendezvous channel, here channel 0, and waits. The transmitter passes through that channel once per cycle of five packets, so contact is re-established within roughly one cycle. This converts total collapse into bounded, periodic loss.

The trigger is a silence timer:

$$t - t_{\text{last}} > T_{\text{r}} = 2000 \text{ ms} \;\Rightarrow\; c_{\text{rx}} \leftarrow c_0, \tag{11}$$

with t_{last} the arrival time of the last accepted packet. Once the receiver is parked on c_0 the transmitter passes it within one cycle, so the nominal time from loss of synchronisation to recovery is

$$T_{\text{acq}} \approx T_{\text{r}} + T_{\text{c}} = 2.0 + 6.0 = 8.0 \text{ s}. \tag{12}$$

This is a typical figure under the measured operating conditions, not a guaranteed bound. It is computed from the median cadence, and individual inter-packet gaps of up to about 4 s were observed in the logs, so a true worst case would be longer. Establishing a guaranteed bound would require the transmit path to be given a deterministic timing budget, which the blocking driver call used here does not provide.

The receiver therefore has only two states, shown in Fig. 4, and Algorithm 2 gives the loop. Note that the receiver never senses and never chooses a channel: all adaptivity lives at the transmitter, so the same receiver firmware serves every configuration in Section IV.

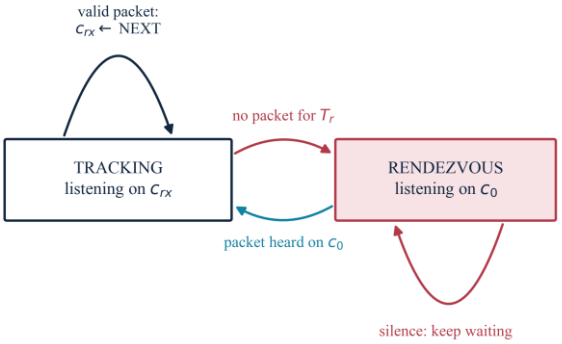


Fig. 4. Receiver schedule tracking. A decoded packet advances the receiver to the announced channel; T_{r} of silence sends it back to the rendezvous channel c_0.

Algorithm 2 Receiver schedule tracking and recovery

```
procedure ReceiveLoop
    c_rx ← c_0;  t_last ← now
    loop
        if PacketReady() and CrcOk() then
            if denied(c_rx) then
                discard packet   ▷ NEXT is lost with it
            else
                (seq, next) ← Parse(payload)
                record seq;  t_last ← now
                c_rx ← next;  tune to f_next
            end if
        else if now − t_last > T_r then
            c_rx ← c_0;  tune to f_0;  t_last ← now
        end if
    end loop
end procedure
```

E. Sense-Before-Hop Channel Avoidance

The adaptive transmitter does not simply advance to the next channel. Before committing it tunes to the candidate, lets the RSSI register settle for 5 ms, and reads the ambient level. A candidate whose level exceeds a threshold Λ is declared occupied, is blacklisted for three cycles, and the next candidate is examined:

$$\text{accept channel } c \;\Leftrightarrow\; P_{\text{RSSI}}(c) < \Lambda,\;\; \Lambda = -75 \text{ dBm}. \tag{13}$$

A refused channel is not banned permanently. Each channel carries a cooldown counter that is set on refusal and aged once per hop cycle,

$$\text{cd}_c \leftarrow 3 \text{ on refusal};\quad \text{cd}_c \leftarrow \max(\text{cd}_c - 1, 0) \text{ each cycle}, \tag{14}$$

so a channel that was briefly busy rejoins the hop set after three cycles with no signalling between the two ends. Writing B for the set of channels currently refused, the usable hop set and the condition for the link to continue at all are

$$|A| = N - |B|, \quad |A| \geq 1 \;\text{ and }\; c_0 \notin B. \tag{15}$$

Both conditions are needed. The first keeps a channel available to carry traffic; the second keeps the rendezvous channel usable, without which a receiver that loses the schedule cannot re-acquire it however many other channels remain. Channel 0 was never denied in any run reported here, so the second condition held throughout and was not tested. This is the sense in which hopping buys a bounded amount of protection: it fails when the whole hop set is denied, which is the wideband case excluded throughout. Algorithm 1, drawn

as Fig. 5, gives the selection procedure, and Algorithm 3 the transmit cycle that calls it. Two design choices are worth drawing out. If every candidate appears occupied the transmitter falls back to plain sequential hopping rather than stalling: a link that keeps trying is more useful than one that stops. And whatever channel is chosen is announced in the NEXT field exactly as the non-adaptive transmitter would announce it, so the receiver needs no modification between the basic and adaptive designs.

Algorithm 1 Sense-before-hop channel selection

```
procedure SelectNextHop(current)
    for offset = 1 to N do
        c ← (current + offset) mod N
        if cooldown[c] > 0 then continue
        tune to f_c; enter RX; wait 5 ms
        p ← ReadRSSI()
        if p < Λ then return c
        cooldown[c] ← 3
    end for
    return (current + 1) mod N   ▷ fallback
end procedure
```

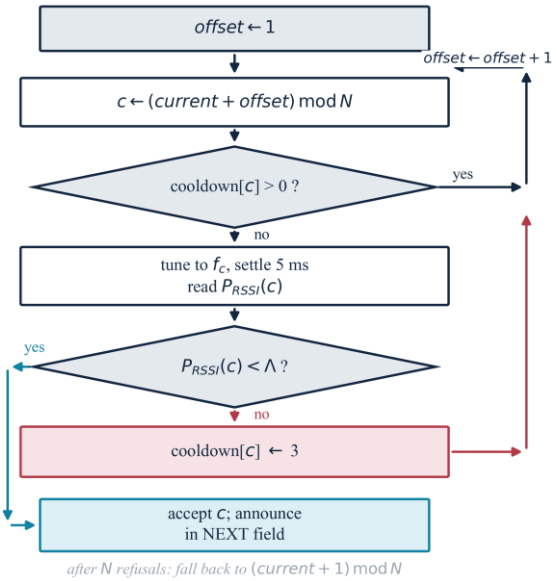


Fig. 5. Sense-before-hop channel selection (Algorithm 1). A candidate is refused either because it is still cooling down or because its measured level exceeds Λ.

Algorithm 3 Adaptive transmit cycle

```
procedure TransmitCycle
    for c = 0 to N − 1 do
        cooldown[c] ← max(cooldown[c] − 1, 0)
    end for
    next ← SelectNextHop(current)   ▷ Algorithm 1
    payload ← "SEQ:⟨seq⟩,NEXT:⟨next⟩"
    transmit payload on f_current
    seq ← (seq + 1) mod M
    current ← next
    wait T_p = 700 ms   ▷ cadence T_i ≈ 1.2 s
end procedure
```

IV. Experimental Method

A. Emulated Channel Denial

No interference is transmitted at any point in this work. Channel denial is emulated in firmware, for three reasons: operating an interference source requires authorisation this project does not hold; a third radio would be needed and only two are available; and emulation is exactly repeatable, so runs are genuinely comparable.

A bitmask marks channels as denied. At the receiver, a packet arriving while tuned to a denied channel is discarded on arrival, taking its NEXT field with it, which reproduces what an unusable channel actually costs. At the adaptive transmitter, a denied channel returns a synthetic −35 dBm when sensed, so the avoidance logic responds as it would to a genuinely occupied one.

The scope of this choice bounds what the results support. Emulation validates the decision and recovery logic under channel denial. It does not validate the RF sensing front end against a real interferer: the only runs in which every sensing reading comes from the radio are those with no denied channels, and run 7 is therefore the sole source of the false-alarm statistic in Section V-F.

B. Experiment Matrix and Metrics

Nine configurations were measured, listed in Table IV. Each sends exactly $M = 100$ numbered packets. The matrix is driven by a single host script that compiles each sketch with the appropriate flags, uploads to the correct serial port, resets the receiver, starts the transmitter, captures both serial streams onto one timeline and parses the receiver's summary. Manual intervention is eliminated, so a run can be repeated exactly.

TABLE IV
Experiment Matrix

Run	Link design	Recovery	Denied	Purpose
1	Fixed	—	0	Control
2	Fixed	—	1	Motivating failure
3	Basic FHSS	off	0	Hopping baseline
4	Basic FHSS	off	1	Sync-loss collapse
5	Basic FHSS	on	1	Recovery bounds loss
6	Basic FHSS	on	2	Scaling with denial
7	Adaptive	on	0	Genuine RSSI sensing
8	Adaptive	on	1	Avoidance
9	Adaptive	on	2	Avoidance, two channels

Channel 2 is denied for one; channels 2 and 4 for two. The rendezvous channel is channel 0 throughout.

The primary metric is packet loss,

$$L = (1 - R/M) \times 100\ \%, \tag{16}$$

where R is the number of **unique** sequence numbers accepted. Reception is recorded in a Boolean array indexed by sequence number and bounds-checked, so a duplicate cannot inflate the count. A second quantity, *denied drops*, counts packets heard and then discarded because the receiver was tuned to a denied channel; these are a subset of losses in which the frame was physically received.

The complementary ratio is the packet delivery ratio,

$$\mathrm{PDR} = R / M = 1 - L/100, \tag{17}$$

so run 8, with $R = 98$ of $M = 100$, gives PDR = 0.98.

One accounting detail is worth stating, because the counts do not look self-consistent otherwise. Run 2 records 101 denied drops against 100 packets sent. The drop counter counts frames physically received and then discarded, and the capture logs show SEQ:0 transmitted twice at the start of a run, because opening the serial port restarts the transmitter after it has already begun. The 101 discarded frames therefore carry 100 distinct sequence numbers. Delivery is counted on distinct sequence numbers throughout, so the duplicate affects no reported loss figure.

Operation in 433.05–434.79 MHz is licence-exempt in India under the 2022 Rules [37], which permit 10 mW effective radiated power at up to a 10 % duty cycle. A frame of $L_f = 22$ bytes occupies

$$T_{on} = 8L_f / R_b = 176 / 99.97\ \text{kb/s} \approx 1.8\ \text{ms}, \tag{18}$$

and at the measured inter-packet interval $T_i = 1.200$ s established in Section III-C, the duty cycle is

$$D = T_{on} / T_i = 1.8 / 1200 \approx 1.5 \times 10^{-3} = 0.15\ \%, \tag{19}$$

roughly two orders of magnitude inside the limit.

V. Results and Discussion

Table V gives the complete measured matrix and Fig. 6 plots loss against the number of denied channels. Table VI gives the per-channel distribution, which is the most diagnostic output. All values are from single runs of 100 packets.

TABLE V
Measured Results, Nine Configurations

Run	Design	Rec.	Den.	*R*	*L* (%)	Drops	Resync
1	Fixed	—	0	100	0.00	0	0
2	Fixed	—	1	0	100.00	101	0
3	Basic	off	0	95	5.00	0	0
4	Basic	off	1	2	98.00	20	0
5	Basic	on	1	40	60.00	20	47
6	Basic	on	2	40	60.00	19	47
7	Adaptive	on	0	100	0.00	0	1
8	Adaptive	on	1	98	2.00	0	2
9	Adaptive	on	2	99	1.00	0	2

Rec. = rendezvous recovery; Den. = denied channels; *R* = unique packets received; Drops = denied drops.

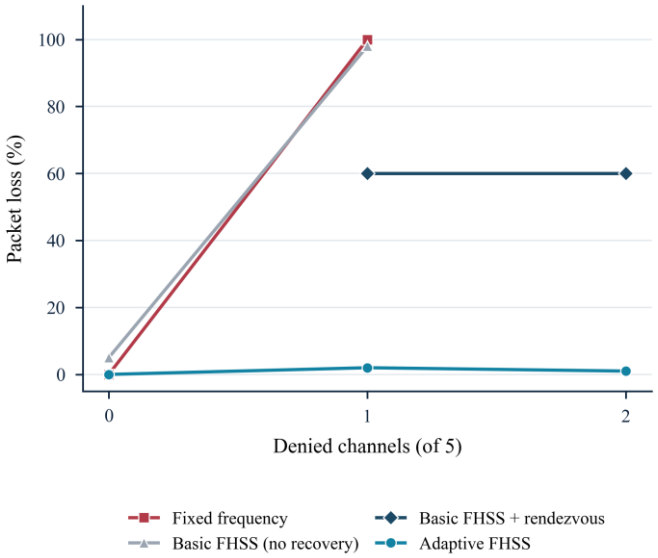


Fig. 6. Packet loss against the number of denied channels. The fixed link and the basic hopping link without recovery both collapse at a single denied channel; the adaptive link does not.

A. Baselines

Run 1 delivered 100 of 100 packets on a fixed channel, establishing that the link, the wiring and the radio configuration are sound. Run 2 denied that one channel and delivered none: 100 % loss, with 101 denied drops confirming that the energy was arriving and being deliberately discarded. This pair is the motivation for the project in two numbers.

Run 3, basic hopping with nothing denied, delivered 95 of 100 with an almost perfectly even distribution of 19 packets on each of the five channels, confirming the hop schedule cycles as intended. The 5 % loss relative to the fixed baseline is the cost of hopping itself: retuning takes time and every hop is an opportunity to lose synchronisation.

B. Collapse of In-Band Scheduling

Run 4 is the central negative result. One channel of five was denied and recovery was disabled. A per-channel argument predicts 20 % loss. The measured loss was **98 %**. Defining the amplification of a partial-band denial as the ratio of measured to predicted loss,

$$A = L_{meas} / L_{naive} = 98 / 20 \approx 4.9. \tag{20}$$

The per-channel tally explains it exactly. The receiver accepted one packet on channel 0 and one on channel 1, followed the schedule onto channel 2, and lost the packet there together with its NEXT field. Stranded on channel 2, it then heard the transmitter only on that channel, once every five packets:

$$2\ \text{accepted} + 20\ \text{discarded} + 78\ \text{unheard} = 100. \tag{21}$$

The denial itself cost 20 packets; loss of synchronisation cost the other 78. With the hop schedule embedded in the data, the dominant damage mechanism is not the denied channel but the desynchronisation it triggers.

C. Recovery and the Saturation Effect

Runs 5 and 6 enable cyclic recovery. Run 5, with one channel denied, lost 60 % instead of 98 %. Run 6, with *two* channels denied, lost 60 % as well — the same figure to two decimal places, with identical per-channel tallies and the same 47 resynchronisation events.

This is not measurement noise. The receiver, having recovered to rendezvous channel 0, walks forward through the schedule until it meets the *first* denied channel, where it is stranded again and the timer returns it to rendezvous. It never reaches any channel beyond the first denied one, so denying channel 4 as well changed nothing.

Let d be the number of hops from the rendezvous channel to the first denied channel in schedule order. Per recovery cycle the receiver delivers d packets out of N. Without recovery it delivers d packets in total and then stops; with recovery it repeats once per cycle. Hence

$$R_{\text{no rec}} = d, \quad R_{\text{rec}} \approx d \cdot M/N. \tag{22}$$

Neither expression contains the number of denied channels K. The model therefore predicts saturation outright,

$$\partial R_{\text{rec}} / \partial K = 0 \quad \text{for} \quad K \geq 1, \tag{23}$$

because the receiver never survives past the first denied channel and so never reaches the second. Runs 5 and 6 differ only in K, and both delivered 40 packets.

The second expression follows from counting cycles. Over M packets the transmitter completes

$$n_c = \lfloor M / N \rfloor = \lfloor 100 / 5 \rfloor = 20 \tag{24}$$

cycles, and each one delivers the d packets that precede the denied channel, giving $R_{\text{rec}} = d \cdot n_c = 2 \times 20 = 40$. Treating the denied channel as equally likely to be any of the $N - 1$ non-rendezvous channels makes d uniform on $\{1, \ldots, N - 1\}$, so that $\mathrm{E}[d] = N/2$ and

$$\mathrm{E}[R_{\text{rec}}] = (N/2)(M/N) = M/2 = 50. \tag{25}$$

Half the stream is lost on average, independently of how many channels the link hops over. Widening the hop set does not help, because it lengthens the cycle in exactly the proportion it increases the number of channels reached before the bad one. This is a prediction of the model rather than a measurement: only $d = 2$ was tested, and Section VI lists varying d as the experiment that would confirm it.

With rendezvous at channel 0 and the first denied channel at index 2, $d = 2$, $N = 5$ and $M = 100$. The model predicts 2 delivered packets for run 4, 40 for run 5 and 40 for run 6. The measured values are 2, 40 and 40. All three are reproduced exactly, which is strong evidence that the mechanism has been correctly identified rather than merely described.

The practical consequence is worth stating: under in-band scheduling with cyclic recovery, link quality is governed by *where* the first unusable channel sits relative to the rendezvous point, not by *how many* channels are unusable. Adding channels to the hop set does not help if the obstruction is early in the order. Avoidance, not recovery, is what addresses this.

TABLE VI
PER-CHANNEL PACKETS RECEIVED

Run	Design	CH0	CH1	CH2	CH3	CH4	Total
3	Basic, 0 denied	19	19	19	19	19	95
4	Basic, 1 denied	1	1	0*	0	0	2
5	Basic + rec., 1 den.	20	20	0*	0	0	40
6	Basic + rec., 2 den.	20	20	0*	0	0*	40
7	Adaptive, 0 denied	20	20	20	20	20	100
8	Adaptive, 1 denied	25	25	0*	24	24	98
9	Adaptive, 2 denied	34	33	0*	32	0*	99

An asterisk marks a denied channel.

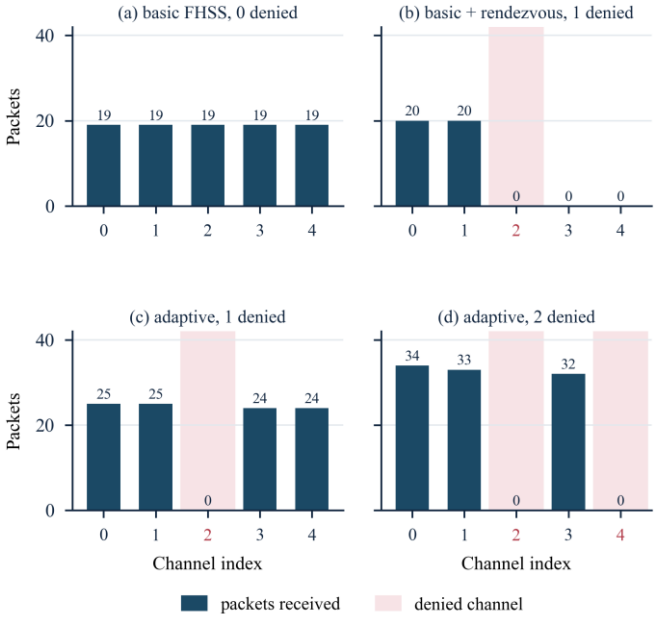


Fig. 7. Per-channel packet distribution. (b) With recovery and one denied channel the receiver never advances past channel 2, which is why denying a second changes nothing. (c), (d) The adaptive transmitter redistributes traffic onto the remaining channels.

D. Adaptive Avoidance

Runs 7 to 9 use the sense-before-hop transmitter. With nothing denied it delivered 100 of 100. With one channel denied it delivered 98, a loss of 2 %. With two denied it delivered 99.

The per-channel tallies in Table VI show the mechanism working directly. In run 8 the denied channel 2 received exactly zero packets while the other four carried 25, 25, 24 and 24 — the traffic was redistributed, not merely lost. In run 9 the two denied channels carried zero and the three remaining channels carried 34, 33 and 32. The transmitter avoided the unusable channels in advance instead of discovering them by failure.

Run 9 showing marginally lower loss than run 8 should not be read as a trend; a difference of one packet in a hundred from single runs is well inside the expected variation. Both are properly described as near-complete avoidance, and the responsible summary is that for a single denied channel, loss falls from 98 % under basic hopping to 2 % under adaptive avoidance.

E. A Channel-Plan Error Found by Measurement

The first version of the design used 200 kHz channel spacing with an 812.50 kHz receive bandwidth. A 195 kHz transmission in a 200 kHz grid very nearly touches its neighbours, and a ±406 kHz passband is *wider than the entire 800 kHz hop span*, so a receiver parked on any single channel still hears most of the others.

This was caught by an anomaly in the data rather than predicted. Under that configuration a stranded receiver was observed picking up roughly half of all transmissions, which is impossible if the channels are isolated. Widening the spacing to 400 kHz and narrowing the filter to 270.83 kHz produced the geometry of (3) and the clean, mutually exclusive per-channel tallies of Table VI. The episode illustrates that a spread-spectrum design can appear to work while delivering none of the isolation it claims.

F. Noise Floor and False-Alarm Rate

Run 7 is the only run in which every sensing reading comes from the radio rather than from emulation. Across 101 genuine channel assessments the measured level was −105.3 ± 1.6 dBm, ranging from −109 to −101 dBm, with per-channel means between −106.3 and −104.3 dBm (Fig. 8). None of the 101 readings reached the −75 dBm threshold, giving a measured false-alarm rate of 0/101 with the threshold roughly 30 dB above the floor.

Comparing the measured floor with the thermal prediction of (8) gives the combined noise figure and implementation loss of the receive chain,

$$F = P_{\text{meas}} - P_{\text{n}} = -105.3 - (-119.7) = 14.4 \text{ dB}, \tag{26}$$

which is unremarkable for a low-cost integrated transceiver. The threshold sits far above the floor; expressed in standard deviations of the measured distribution the margin is

$$(\Lambda - \mu) / \sigma = (-75 + 105.3) / 1.6 \approx 18.9\ \sigma. \tag{27}$$

No false alarm was therefore expected and none occurred. With zero events in $n = 101$ trials the rule of three places a 95 % upper confidence bound on the false-alarm probability at

$$P_{\text{fa}} < 3 / n = 3 / 101 \approx 0.030, \tag{28}$$

so the measurement bounds the false-alarm rate below about 3 %; it does not establish that it is zero.

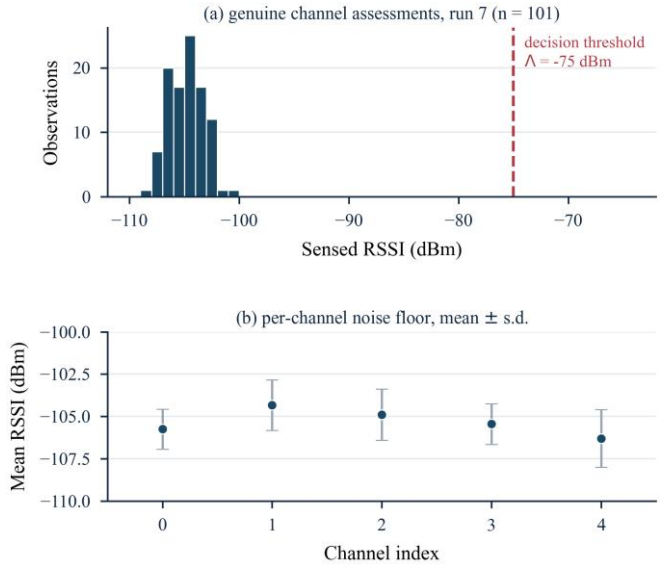


Fig. 8. Measured noise floor from run 7. (a) Distribution of all 101 assessments against the −75 dBm threshold. (b) Per-channel mean and standard deviation; the five channels agree to within 2 dB.

This establishes that the threshold does not fire on the ambient laboratory environment. It says nothing about the detection rate against a real interferer, which emulation cannot measure.

Sensing costs about 5 ms per packet with no denial, rising to roughly 8.5 ms with two denied channels, because a cooldown suppresses repeated testing of a channel already known to be bad. The end-to-end packet cadence was 1.215 s for basic hopping and 1.221 s for adaptive, a difference well inside the variation of the serial logging that dominates both.

G. Ground-Station Dashboard

A host-side dashboard reads both serial streams and serves a live view over HTTP. It renders the five-channel map with the current transmit and receive channels, per-channel reception counts and sensed levels, running delivery and loss statistics, a rolling loss trace and an event feed. Two controls are interactive: clicking a channel denies or releases it at run time, and a mode toggle pins the transmitter to a single channel so the fixed-frequency failure can be shown beside the hopping link without reflashing. Nothing in the dashboard transmits interference.

[INSERT DASHBOARD SCREENSHOT HERE]

Capture the browser window at http://localhost:8080 while the demonstration firmware is running and one channel has been denied. Make sure the channel map, the statistics and the event feed are visible. Save at 300 dpi or greater.

Fig. 9. Live ground-station dashboard during a run with one channel denied by operator action.

[INSERT HARDWARE PHOTOGRAPH HERE]

Photograph both assembled nodes side by side on a plain background, with the wiring and antennas clearly visible. IEEE requires photographs at 300 dpi minimum.

Fig. 10. The two-node prototype: ESP32 DevKit v1 and CC1101 transceiver per node.

VI. Limitations

The following bound what the results support, and are stated here rather than left to be discovered.

1) *Denial is emulated in firmware, not transmitted.* This validates decision and recovery logic. It does not validate the RF sensing front end against a real interferer.
2) *The delivery model is validated at a single operating point.* All three counts reproduced by (22) come from the same hop set, the same rendezvous channel and the same distance $d = 2$. That delivery scales with d and is independent of K follows from the mechanism and is consistent with three measurements, but it is not a swept parameter. Denying channel 1 ($d = 1$) and channel 3 ($d = 3$) and comparing against the prediction is the single experiment that would most strengthen this work, and it requires no additional hardware.
3) *Single trials.* Every entry in Table V is one run of 100 packets; no variance is reported. Differences of one or two packets between runs carry no weight.
4) *A five-channel hop set is small.* Operational systems use tens of channels. With five, denying two removes 40 % of the spectrum.
5) *The hop order is fixed and predictable,* and the rendezvous channel is a single point of failure. Deriving the sequence from a shared secret, as in [1], and rotating the rendezvous channel are the natural next steps.
6) *The channel plan overruns the band edge.* With the occupied bandwidth of (1), five channels at 400 kHz spacing require 4(400) + 195 = 1795 kHz, whereas 433.05–434.79 MHz provides 1740 kHz. The plan does not fit the band at any placement, and as placed the lower skirt of channel 0 extends about 97.5 kHz below the lower edge. A compliant five-channel plan at this bandwidth needs a spacing of at most 386 kHz. This was found in review, after the measurements were taken; it does not affect the protocol results, which concern scheduling rather than emission, but the regulatory claim in Section IV should be read as conducted power only. Relatedly, +10 dBm at the transmitter output is a conducted figure: effective radiated power depends on antenna gain and feed loss, neither of which was measured.
7) *Benchtop conditions,* and no claim against wideband denial. Frequency hopping helps when interference affects part of the hop set; against energy covering the whole band it has nowhere to go.

VII. Conclusion

A complete adaptive frequency-hopping telemetry link was built from two ESP32 microcontrollers and two CC1101 transceivers and measured across nine configurations under firmware-emulated channel denial, with no interference transmitted at any point.

Carrying the hop schedule inside the data packets is economical but fails abruptly: one denied channel of five produced 98 % loss rather than the 20 % a per-channel argument suggests, because 78 of the 100 packets were lost to desynchronisation rather than to denial. Adding cyclic recovery bounds the damage but makes it saturate at a value set by the distance from the rendezvous channel to the first denied channel, not by the number of channels denied; a model built on that single parameter reproduced the measured delivery counts of 2, 40 and 40 exactly. Transmitter-side clear-channel assessment removed the failure mode rather than bounding it, reducing loss for a single denied channel from 98 % to 2 %, with zero false alarms across 101 genuine assessments against a measured floor of -105.3 ± 1.6 dBm.

A design error found only by measurement is reported alongside these results: a receive bandwidth wider than the whole hop span, which left the channels non-independent while the link appeared to function.

Future work falls into three parts: validation against a real interference source in a conducted, shielded setup; a hop sequence derived from a shared secret, with a rotating rendezvous channel; and carrying real MAVLink telemetry in the position a commercial telemetry radio occupies between flight controller and ground station.

Acknowledgment

The authors thank Velmurugan T. for supervision and guidance throughout the project, and the School of Electronics Engineering, Vellore Institute of Technology, for laboratory access.

References

[1] C. Atheeq, Z. Gulzar, M. S. Al Reshan, H. Alshahrani, A. Sulaiman, and A. Shaikh, “Securing UAV networks: A lightweight chaotic-frequency hopping approach to counter jamming attacks,” *IEEE Access*, vol. 12, pp. 38685–38699, 2024, doi: 10.1109/ACCESS.2024.3375343.

[2] Y.-B. Chang, F.-Y. Li, and M.-J. Tsai, “Time-efficient channel hopping sequence with maximum rendezvous diversity in cognitive radio networks,” *IEEE Trans. Commun.*, vol. 74, no. 1, pp. 291–306, Jan. 2026, doi: 10.1109/TCOMM.2025.3624171.

[3] Z. He, S. Feng, C. Qu, B. Tang, C. Pei, and L. Bai, “Sliding window sub-channel detection based on adaptive threshold for high-speed frequency hopping signals,” *IEEE Access*, vol. 14, pp. 44562–44574, 2026, doi: 10.1109/ACCESS.2026.3674783.

[4] E. Jo and H. Baek, “Channel restoration scheme to improve performance of adaptive-fast frequency hopping remapping,” *J. Korean Inst. Commun. Inf. Sci.*, vol. 51, no. 1, pp. 232–240, Jan. 2026, doi: 10.7840/kics.2026.51.1.232.

[5] C. Oh and S. Kang, “LSTM-based early jamming threat detection scheme for drone ad-hoc networks,” *Appl. Sci.*, vol. 16, no. 6, p. 3046, 2026, doi: 10.3390/app16063046.

[6] O. M. Ayegun, K. F. Akingbade, J. J. Popoola, and B. C. Ubochi, “Leveraging on artificial intelligence for universal jamming attack detection in unmanned aerial vehicles,” *Comput. Electr. Eng.*, vol. 127, p. 110602, 2025, doi: 10.1016/j.compeleceng.2025.110602.

[7] S. Cheng, X. Ling, and L. Zhu, “Deep reinforcement learning-based anti-jamming approach for fast frequency hopping systems,” *IEEE Open J. Commun. Soc.*, vol. 6, pp. 961–971, 2025, doi: 10.1109/OJCOMS.2025.3529982.

[8] S. Cibecchini, F. Chiti, and L. Pierucci, “A lightweight AI-based approach for drone jamming detection,” *Future Internet*, vol. 17, no. 1, p. 14, 2025, doi: 10.3390/fi17010014.

[9] G. Hochet Derevianckine, A. Guitton, O. Iova, B. Ning, and F. Valois, "Hate or love in the 2.4 GHz ISM band: The story of LoRa and IEEE 802.11g," *ACM Trans. Internet Things*, vol. 7, pp. 1–30, 2025, doi: 10.1145/3743688.

[10] L. Song, D. Guo, T. Hu, and J. Yang, "Deep reinforcement learning-based anti-jamming rate adaptation and frequency selection algorithm," *IEEE Trans. Consum. Electron.*, vol. 71, pp. 11782–11790, 2025, doi: 10.1109/TCE.2025.3627267.

[11] M. Asadi Vasfi and B. Shahgholi Ghahfarokhi, "Channel-hopping sequence generation for blind rendezvous in cognitive radio-enabled Internet of Things," *Comput. Commun.*, vol. 243, p. 108318, 2025, doi: 10.1016/j.comcom.2025.108318.

[12] S. Wang, D. Liu, L. Yin, X. Wang, Z. Ke, and Z. Du, "Dual-mode anti-jamming strategy for UAV communications," in *Proc. 8th Int. Conf. Electron. Technol. (ICET)*, Wuhan, China, 2025, pp. 131–136, doi: 10.1109/ICET64964.2025.11102942.

[13] W. Xie, H. Yang, and Z. Xiong, "Resource allocation for UAV-assisted anti-jamming semantic D2D networks: A graph reinforcement learning approach," *Comput. Netw.*, vol. 269, p. 111463, 2025, doi: 10.1016/j.comnet.2025.111463.

[14] J. Jiao, C. Liu, J. He, and M. Peng, "Deep reinforcement learning assisted trajectory design for UAV communication systems against malicious jamming," in *Proc. IEEE 23rd Int. Conf. Commun. Technol. (ICCT)*, Wuxi, China, 2023, pp. 979–984, doi: 10.1109/ICCT59356.2023.10419848.

[15] G. Cena, S. Scanzio, and A. Valenzano, "Ultra-low power wireless sensor networks based on time slotted channel hopping with probabilistic blacklisting," *Electronics*, vol. 11, no. 3, p. 304, Jan. 2022, doi: 10.3390/electronics11030304.

[16] H. Pirayesh and H. Zeng, "Jamming attacks and anti-jamming strategies in wireless networks: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 2, pp. 767–809, 2nd Quart. 2022, doi: 10.1109/COMST.2022.3159185.

[17] A. Allouch, O. Cheikhrouhou, A. Koubaa, M. Khalgui, and T. Abbes, "MAVSec: Securing the MAVLink protocol for Ardupilot/PX4 unmanned aerial systems," in *Proc. 15th Int. Wireless Commun. Mobile Comput. Conf. (IWCMC)*, Tangier, Morocco, 2019, pp. 621–628, doi: 10.1109/IWCMC.2019.8766667.

[18] V. Kotsiou, G. Z. Papadopoulos, D. Zorbas, P. Chatzimisios, and F. Théoleyre, "Blacklisting-based channel hopping approaches in low-power and lossy networks," *IEEE Commun. Mag.*, vol. 57, no. 2, pp. 48–53, Feb. 2019, doi: 10.1109/MCOM.2018.1800362.

[19] A. Koubaa, A. Allouch, M. Alajlan, Y. Javed, A. Belghith, and M. Khalgui, "Micro air vehicle link (MAVlink) in a nutshell: A survey," *IEEE Access*, vol. 7, pp. 87658–87680, 2019, doi: 10.1109/ACCESS.2019.2924410.

[20] X. J. Tan and Y. Yuan, "Efficient channel hopping for blind rendezvous in multi-radio cognitive radio networks," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Waikoloa, HI, USA, 2019, pp. 1–6, doi: 10.1109/GLOBECOM38437.2019.9014326.

[21] X. Liu, Y. Xu, L. Jia, Q. Wu, and A. Anpalagan, "Anti-jamming communications using spectrum waterfall: A deep reinforcement learning approach," *IEEE Commun. Lett.*, vol. 22, no. 5, pp. 998–1001, May 2018, doi: 10.1109/LCOMM.2018.2815018.

[22] A. Elsts, X. Fafoutis, R. Piechocki, and I. Craddock, "Adaptive channel selection in IEEE 802.15.4 TSCH networks," in *Proc. Global Internet Things Summit (GIoTS)*, Geneva, Switzerland, 2017, pp. 1–6, doi: 10.1109/GIOTS.2017.8016246.

[23] S. Jaitly, H. Malhotra, and B. Bhushan, "Security vulnerabilities and countermeasures against jamming attacks in wireless sensor networks: A survey," in *Proc. Int. Conf. Comput., Commun. Electron. (Comptelix)*, Jaipur, India, 2017, pp. 559–564, doi: 10.1109/COMPTELIX.2017.8004033.

[24] S. Lv, L. Xiao, Q. Hu, X. Wang, C. Hu, and L. Sun, "Anti-jamming power control game in unmanned aerial vehicle networks," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Singapore, 2017, pp. 1–6, doi: 10.1109/GLOCOM.2017.8253988.

[25] L. Petnga and H. Xu, "Security of unmanned aerial vehicles: Dynamic state estimation under cyber-physical attacks," in *Proc. Int. Conf. Unmanned Aircraft Syst. (ICUAS)*, Arlington, VA, USA, 2016, pp. 811–819, doi: 10.1109/ICUAS.2016.7502663.

[26] Y. Song, K. W. Sung, and Y. Han, "Coexistence of Wi-Fi and cellular with listen-before-talk in unlicensed spectrum," *IEEE Commun. Lett.*, vol. 20, no. 1, pp. 161–164, Jan. 2016, doi: 10.1109/LCOMM.2015.2504509.

[27] M. R. Palattella and L. A. Grieco, *Using IEEE 802.15.4e Time-Slotted Channel Hopping (TSCH) in the Internet of Things (IoT): Problem Statement*, RFC 7554, May 2015, doi: 10.17487/RFC7554.

[28] Z. Lin, H. Liu, X. Chu, and Y.-W. Leung, "Jump-stay based channel-hopping algorithm with guaranteed rendezvous for cognitive radio networks," in *Proc. IEEE INFOCOM*, Shanghai, China, 2011, pp. 2444–2452, doi: 10.1109/INFCOM.2011.5935066.

[29] T. Yucek and H. Arslan, "A survey of spectrum sensing algorithms for cognitive radio applications," *IEEE Commun. Surveys Tuts.*, vol. 11, no. 1, pp. 116–130, 1st Quart. 2009, doi: 10.1109/SURV.2009.090109.

[30] S. Moon, "An adaptive frequency hopping method in the Bluetooth baseband," in *Proc. 1st Int. Symp. Wireless Pervasive Comput. (ISWPC)*, Phuket, Thailand, 2006, pp. 1–4, doi: 10.1109/ISWPC.2006.1613638.

[31] A. Sikora and V. F. Groza, "Coexistence of IEEE 802.15.4 with other systems in the 2.4 GHz-ISM-band," in *Proc. IEEE Instrum. Meas. Technol. Conf. (IMTC)*, Ottawa, ON, Canada, 2005, vol. 3, pp. 1786–1791, doi: 10.1109/IMTC.2005.1604479.

[32] W. Xu, W. Trappe, Y. Zhang, and T. Wood, "The feasibility of launching and detecting jamming attacks in wireless networks," in *Proc. 6th ACM Int. Symp. Mobile Ad Hoc Netw. Comput. (MobiHoc)*, Urbana-Champaign, IL, USA, 2005, pp. 46–57, doi: 10.1145/1062689.1062697.

[33] Y. Kim, B. Zhen, and K. Jang, "The hybrid of listen-before-talk and adaptive frequency hopping for coexistence of Bluetooth and IEEE 802.11 WLAN," in *Wireless LANs and Home Networks*. Singapore: World Scientific, 2001, pp. 263–272, doi: 10.1142/9789812799562_0024.

[34] C. M. Lo, E. Masry, and L. B. Milstein, "Performance analysis of fast frequency-hopped DBPSK spread spectrum communications in partial-band noise jamming," in *Proc. IEEE Mil. Commun. Conf. (MILCOM)*, McLean, VA, USA, 1991, pp. 576–580, doi: 10.1109/MILCOM.1991.258321.

[35] R. L. Pickholtz, D. L. Schilling, and L. B. Milstein, "Theory of spread-spectrum communications — A tutorial," *IEEE Trans. Commun.*, vol. 30, no. 5, pp. 855–884, May 1982, doi: 10.1109/TCOM.1982.1095533.

[36] *ESP32 Series Datasheet*, version 5.3, Espressif Systems, Shanghai, China. Accessed: Oct. 15, 2026. [Online]. Available: https://documentation.espressif.com/esp32_datasheet_en.pdf

[37] *Use of Low Power Radio Frequency Devices in the Frequency Band 433.05 to 434.79 MHz (Exemption from Licence) Rules, 2022*, G.S.R. 347(E), Ministry of Communications, Government of India, Gazette of India, May 9, 2022.

[38] *CC1101 Low-Power Sub-1 GHz RF Transceiver*, Texas Instruments, Dallas, TX, USA, Datasheet SWRS061I, Nov. 2013.

[39] P. E. Voglewede, E. M. Leiby III, M. W. Chamberlain, P. Lang, F. C. Kellerman, and D. E. Stephens, "Signal jamming avoidance method for a frequency hopping communication system," U.S. Patent 8 165 242 B2, Apr. 24, 2012.

[40] R. R. Marcoccia and G. R. Sosnowski, "Method and transceiver using an improved protocol for a frequency hop communication system," U.S. Patent 6 169 761 B1, Jan. 2, 2001.